CS306 – PHASE III REPORT
Web Integrations of SQL and NoSQL Databases

Group Members:

- Cem Sarp Takım 32245
- Petek Metin 32627
- Ege Derman 32246
- Başar Erses 31904

---

1. Introduction

This document presents the final phase of our **Second-Hand Marketplace** project, detailing the database triggers, stored procedures, and MongoDB-based support ticket system. Each backend feature is mapped to corresponding frontend components for demonstration purposes.

2. Trigger Integrations

2.1) before_insert_vehicle_price_check

Purpose:
This trigger ensures that vehicle listings cannot be submitted with unreasonably low prices. To prevent unreal or fake listings. Price must be bigger than 1000.

SQL Script:

```
DELIMITER $$

CREATE TRIGGER `before_insert_vehicle_price_check`
BEFORE INSERT ON `Vehicles`
FOR EACH ROW
BEGIN
   DECLARE price_amt DECIMAL(10,2);

   SELECT p.amount INTO price_amt
   FROM Prices p
   JOIN Listings l ON l.price_id = p.price_id
   WHERE l.lid = NEW.lid
   LIMIT 1;

   IF price_amt < 1000 THEN
```

```
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Price is suspiciously low for a vehicle. Must be at least 1000.';
    END IF;
END$$

DELIMITER ;
```

Web page buttons:

Scenario	Button Label	Behavior	Result
Valid Insert	`Test Valid Insert (price = 5000)`	Allows insert	✅ “Successfully inserted”
Invalid Price	`Test Invalid Insert (price = 200)`	Blocks insert (trigger fires)	❌ “Trigger blocked insert: Price is not valid must be bigger than 1000”

2.2) before_insert_electronics_spec_check

Purpose:
This trigger ensures that electronics listings cannot be submitted without essential specifications such as insurance type, device age, or brand.

SQL Script:

```
DELIMITER $$

CREATE TRIGGER `before_insert_electronics_spec_check`
BEFORE INSERT ON `Electronics`
FOR EACH ROW
BEGIN
    IF NEW.insurance IS NULL
        OR NEW.eage IS NULL
        OR NEW.ebrand IS NULL
        OR NEW.ebrand = '' THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Missing required specifications for electronics.';
    END IF;
END$$

DELIMITER ;
```

Web page buttons:

Scenario	Button Label	Behavior	Result
Valid Specifications	`Test VALID Insert (all specs)`	Allows insert	✅ “Valid insert succeeded (trigger allowed).”
Missing Specifications	`Test INVALID Insert (missing specs)`	Blocks insert (trigger fires)	❌ “Trigger blocked insert: Missing required specifications for electronics.”

2.3) before_insert_house_specs

Purpose:
Ensures that a house listing cannot be inserted if any of the following fields are `NULL`: `m2`, `room_count`, or `bage`. This maintains data quality and prevents incomplete listings.

SQL Script:

```
DELIMITER $$

CREATE TRIGGER `before_insert_house_specs`
BEFORE INSERT ON `Houses`
FOR EACH ROW
BEGIN
  IF NEW.m2 IS NULL OR NEW.room_count IS NULL OR NEW.bage IS NULL THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Missing required specifications';
  END IF;
END$$

DELIMITER ;
```

Web page buttons:

Scenario	Button Label	Behavior	Result
Valid Insert	`Test Valid Insert (all specs)`	Allows insert	✅ “Valid insert succeeded! House ... was added”
Invalid Specs	`Test Invalid Insert (missing specs)`	Blocks insert (trigger fires)	❌ “Trigger blocked insert: Missing required specifications”

2.4) before_insert_user_specs

Purpose:
Ensures that a user cannot be inserted without a valid phone number. This helps prevent incomplete or invalid user entries during registration
SQL Script:

```
DELIMITER $$

CREATE TRIGGER `before_insert_user_specs`
BEFORE INSERT ON `Users`
FOR EACH ROW
BEGIN
   IF NEW.phone_number IS NULL OR NEW.phone_number = '' THEN
      SIGNAL SQLSTATE '45000'
         SET MESSAGE_TEXT = 'Phone number is required.';
   END IF;
END$$

DELIMITER ;
```

Web page buttons:

Scenario	Button Label	Behavior	Result
Valid Insert	Submit Valid User	Allows insert	✅ “User inserted successfully.”
Missing Phone	Submit Without Phone	Blocks insert (trigger fires)	❌ “Trigger blocked insert: Phone number is required.”

3. Stored Procedures

3.1) sp_create_vehicle_listing

Purpose:
This stored procedure is used to create a new vehicle listing. It links the Listings, Vehicles, and Prices tables by handling foreign key constraints and generating a unique listing ID automatically. It simplifies the backend logic and ensures data consistency across related tables.

SQL Script:

```
DELIMITER $$
CREATE DEFINER=`root`@`localhost` PROCEDURE `sp_create_vehicle_listing`(IN `p_userid` INT, IN `p_headline` VARCHAR(200), IN `p_description` TEXT, IN `p_location_id` INT, IN `p_date_id` INT, IN `p_price_id` INT, IN `p_km` INT, IN `p_vbrand` VARCHAR(50), IN `p_vage` INT)
BEGIN
   DECLARE new_lid INT;

   -- First, find the maximum lid and increment it
   SELECT COALESCE(MAX(lid), 0) + 1 INTO new_lid FROM Listings;

   -- Insert into Listings table with the new lid
   INSERT INTO Listings (lid, userid, headline, description, location_id, date_id, price_id)
   VALUES (new_lid, p_userid, p_headline, p_description, p_location_id, p_date_id, p_price_id);

   -- Insert into Vehicles table
   INSERT INTO Vehicles (lid, category_id, km, vbrand, vage)
   VALUES (new_lid, 1, p_km, p_vbrand, p_vage);

   -- Return the new lid for reference
   SELECT new_lid AS new_listing_id;
END$$
DELIMITER ;
```

Web page:

Parameter	Form Field Name	Input Type
p_userid	(session)	hidden
p_headline	`Headline`	text
p_description	`Description`	textarea
p_location_id	derived from city/street	hidden
p_date_id	auto-generated	hidden
p_price_id	auto-generated	hidden
p_vbrand	`Brand`	select from list
p_km	`Kilometers`	number
p_vage	`Age(years)`	number
price	`Price`	number
currency	`currency`	select from list

The system automatically generates the **date_id** and **price_id**, and resolves the **location_id** based on the selected city and district before invoking the stored procedure.

3.2) sp_generate_listing_report

Purpose:

This procedure takes a category and outputs matching listings with relevant user and category-specific details.

SQL Script:

```
DELIMITER$$
CREATE DEFINER=`root`@`localhost` PROCEDURE `sp_generate_listing_report` (IN
`p_category` VARCHAR(50))   BEGIN
   IF p_category = 'Vehicle' THEN
      SELECT L.lid, U.name, L.headline, V.km, V.vbrand, V.vage
      FROM Listings L
      JOIN Users U ON L.userid = U.userid
      JOIN Vehicles V ON L.lid = V.lid;
```

```
    ELSEIF p_category = 'House' THEN
        SELECT L.lid, U.name, L.headline, H.m2, H.room_count, H.bage
        FROM Listings L
        JOIN Users U ON L.userid = U.userid
        JOIN Houses H ON L.lid = H.lid;
    ELSEIF p_category = 'Electronics' THEN
        SELECT L.lid, U.name, L.headline, E.insurance, E.eage, E.ebrand
        FROM Listings L
        JOIN Users U ON L.userid = U.userid
        JOIN Electronics E ON L.lid = E.lid;
    ELSE
        SELECT 'Invalid category' AS Error;
    END IF;
END$$

DELIMITER ;
```

Web page:

Input Field	UI Element
`category`	Dropdown: Category Type Options: `Vehicle`, `Electronics`, `House`
—	Button: Generate Report

The page:

- Provides a dropdown to select the category.
- Submits a POST request to call `sp_generate_listing_report(p_category)`.
- Dynamically displays a table depending on the selected category:
    - Vehicle → `km`, `brand`, `age`
    - House → `m²`, `room_count`, `building age`
    - Electronics → `insurance`, `device age`, `brand`
- If the category is invalid or not selected, it displays: `'Invalid category'`.

3.3) sp_create_electronics_listing

Creates an electronics listing and automatically links it with the listings, prices and electronics tables. Ensures consistency with foreign key constraints and avoids duplicate insertions.

SQL Script:

```
DELIMITER $$

CREATE DEFINER=`root`@`localhost` PROCEDURE `sp_create_electronics_listing`(
    IN `p_userid` INT,
    IN `p_headline` VARCHAR(200),
    IN `p_description` TEXT,
    IN `p_location_id` INT,
    IN `p_date_id` INT,
    IN `p_price_id` INT,
    IN `p_insurance` VARCHAR(50),
    IN `p_eage` INT,
    IN `p_ebrand` VARCHAR(50)
)
BEGIN
    DECLARE new_lid INT;

    -- Insert into Listings table
    INSERT INTO Listings (userid, headline, description, location_id, date_id, price_id)
    VALUES (p_userid, p_headline, p_description, p_location_id, p_date_id, p_price_id);

    SET new_lid = LAST_INSERT_ID();

    -- Insert into Electronics table
    INSERT INTO Electronics (lid, category_id, insurance, eage, ebrand)
    VALUES (new_lid, 3, p_insurance, p_eage, p_ebrand);
END$$

DELIMITER ;

 AS new_listing_id;
END$$
DELIMITER ;
```

Web page:

Input Field	UI Element
`headline`	Text input
`description`	Textarea

`city, street`	Dropdowns
`insurance`	Dropdown or text input
`eage`	Numeric input
`ebrand`	Dropdown or text input
`price_amount`	Numeric input
`currency`	Dropdown

The page auto-inserts `date_id`, `price_id`, and uses the location fields before calling the stored procedure.

3.4) `sp_create_house_listing`

Purpose: Creates a new house listing in the system. It first inserts into `Listings`, then inserts the corresponding house details into `Houses` with a fixed category.

SQL Script:

```
DELIMITER $$

CREATE PROCEDURE sp_create_house_listing(
  IN p_userid INT,
  IN p_headline VARCHAR(255),
  IN p_description TEXT,
  IN p_location_id INT,
  IN p_date_id INT,
  IN p_price_id INT,
  IN p_m2 DECIMAL(10,2),
  IN p_room_count INT,
  IN p_bage INT
)
BEGIN
  INSERT INTO Listings(userid, headline, description, location_id, date_id, price_id)
  VALUES(p_userid, p_headline, p_description, p_location_id, p_date_id, p_price_id);
```

```
  SET @lid = LAST_INSERT_ID();

  INSERT INTO Houses(lid, category_id, m2, room_count, bage)
  VALUES(@lid, 1, p_m2, p_room_count, p_bage);
END$$

DELIMITER ;
```

Web Form Fields:

Parameter	Form Field Name	Input Type
p_userid	(session)	hidden
p_headline	`headline`	text
p_description	`description`	textarea
p_location_id	derived from city/street	hidden
p_date_id	auto-generated	hidden
p_price_id	auto-generated	hidden
p_m2	`m2`	number
p_room_count	`rooms`	number
p_bage	`bage`	number
price	`price_amount`	number
currency	`currency`	select

4. MongoDB Support Ticket System

Tickets are stored in `marketplace.tickets`. Operations are handled via PHP using MongoDB's Driver.

4.1) Connecting to MongoDB

File: mongo_connect.php

Purpose:

Establishes a connection to the MongoDB server and initializes the $manager object for executing queries. Also specifies the collection name used in other scripts.

4.2) Creating a Support Ticket

File: support_create.php
MongoDB Operation:
Inserts a new support ticket into the collection.

Query:

```
$bulk = new MongoDB\Driver\BulkWrite;
$ticketId = new MongoDB\BSON\ObjectId();
$bulk->insert([
    '_id'       => $ticketId,
    'title'     => $title,
    'body'      => $body,
    'email'     => $email,
    'status'    => 'open',
    'created_at'=> date("Y-m-d H:i:s"),
    'comments'  => []
]);
$manager->executeBulkWrite($ticketsCollection, $bulk);
```

4.3) Listing All Support Tickets

File: `support_list.php`
MongoDB Operation:
Fetches all tickets to display in a list view.

Query:

```
$query = new MongoDB\Driver\Query([]);
$cursor = $manager->executeQuery($ticketsCollection, $query);
```

4.4) Viewing a Single Ticket with Details

File: `support_view.php`
MongoDB Operation:
Retrieves a specific ticket by its `_id` and allows adding a comment.

Query:

```
$ticketId = new MongoDB\BSON\ObjectId($_GET['id']);
$query = new MongoDB\Driver\Query(['_id' => $ticketId]);
$ticket = current($manager->executeQuery($ticketsCollection, $query)->toArray());
```

Adding a Comment:

```
$bulk = new MongoDB\Driver\BulkWrite;
$bulk->update(
    ['_id' => $ticketId],
    ['$push' => ['comments' => $newComment]]
);
$manager->executeBulkWrite($ticketsCollection, $bulk);
```

4.5) Marking a Ticket as Closed

File: `support_confirm.php`
MongoDB Operation:
Updates the ticket's status to "closed".

Query:

```
$ticketId = new MongoDB\BSON\ObjectId($_GET['id']);
$bulk = new MongoDB\Driver\BulkWrite;
$bulk->update(
    ['_id' => $ticketId],
    ['$set' => ['status' => 'closed']]
);
$manager->executeBulkWrite($ticketsCollection, $bulk);
```